

UEFI Firmware: OVMF & EDK2

The Default QEMU Firmware: SeaBIOS

When you launch QEMU without explicitly specifying firmware, it boots using SeaBIOS, a lightweight, open-source implementation of the legacy BIOS (Basic Input/Output System). SeaBIOS was designed for simplicity and broad compatibility, but it carries a fundamental architectural limitation for CXL emulation.

What SeaBIOS Cannot Do

- SeaBIOS cannot generate the CXL Early Discovery Table (CEDT), the ACPI table that describes the CXL host bridge topology, Fixed Memory Windows (FMW), and CFMWS (CXL Fixed Memory Window Structures) to the operating system.
- Without the CEDT, the guest OS kernel even with full CXL subsystem support compiled in cannot discover, enumerate, or interact with any CXL device presented by QEMU.

What CXL Actually Requires from Firmware

The CXL 3.1 specification mandates that the platform firmware (BIOS/UEFI) perform the following during boot, before the OS kernel ever runs:

- Enumerate all CXL Host Bridges and Root Ports in the ACPI namespace.
- Build and publish the CEDT (CXL Early Discovery Table) so the OS can find all CXL Fixed Memory Windows.
- Configure the CXL ACPI device objects (`_HID`, `_CRS`, `_CCA`) for each component in the hierarchy.
- Establish the memory map entries for CXL interleaved regions so the OS memory hotplug subsystem can bring them online.

OVMF: The Solution

OVMF (Open Virtual Machine Firmware) is a UEFI firmware implementation derived from EDK2 (EFI Development Kit II) and designed specifically for virtual machines. Unlike SeaBIOS, OVMF:

- Implements the full UEFI 2.x specification, including the ACPI table generation framework.
- Generates a complete and correct CEDT at boot time when QEMU presents CXL hardware via the `pxb-cxl`, `cxl-rp`, and `cxl-type3` device models.
- Supports the UEFI memory map model required for CXL capacity to appear as hotpluggable memory regions.
- Provides the UEFI runtime services (variable store via `OVMF_VARS.fd`) that allow the guest OS to interact with firmware configuration.

2. What We Were Doing Before vs. What We Do Now

Before: Using the Pre-Built APT Package

The original automation script obtained OVMF firmware by installing the ovmf APT package maintained by Ubuntu:

```
madhu@Madhur:~$ sudo apt-get install -y |ovmf
```

After installation, the script simply copied the pre-built binary files from the system path into the CXL workspace:

```
madhu@Madhur:~$ cp /usr/share/OVMF/OVMF_CODE*.fd .
cp /usr/share/OVMF/OVMF_VARS*.fd .

mv OVMF_CODE_4M.fd OVMF_CODE.fd 2>/dev/null || true
mv OVMF_VARS_4M.fd OVMF_VARS.fd 2>/dev/null || true
```

Now: Building OVMF from EDK2 Source

The updated script clones the EDK2 repository directly from the Tianocore upstream, compiles the OVMF firmware from source, and installs the resulting binaries into the workspace. This gives complete control over every aspect of the firmware binary that QEMU loads.

The new process in brief:

- Clone the EDK2 repository with all submodules (CryptoPkg/OpenSSL, MbedTLS, etc.).
- Compile BaseTools — the host-side build toolchain (GenFv, GenFfs, etc.) that the edk2 build command depends on.
- Source edksetup.sh with --reconfig to generate fresh Conf/ files from templates, ensuring the correct toolchain definition is always present.
- Run the build command with the exact same flags Ubuntu uses for its no-secboot 4 MB variant (derived from inspecting debian/rules).
- Copy the resulting OVMF_CODE.fd and OVMF_VARS.fd into the workspace images directory.

Aspect	Before (APT Package)	Now (EDK2 Source Build)
Source	Ubuntu binary package	Tianocore upstream git
Build flags visible	No	Yes, fully controlled
Customisable	No	Yes
Reproducible	Tied to Ubuntu repo state	Pinned git commit
Offline capable	No (requires network/apt)	Yes (after initial clone)
Patch-able	No	Yes, apply any patch before build

3. The debian/rules Build Configuration

How We Obtained This Information

To understand exactly what flags the Ubuntu ovmf binary was compiled with, we used apt-get source to download and unpack the edk2 source package, then read the debian/rules Makefile that Canonical's build servers use. At its core, the debian/rules file is an **executable Makefile** that serves as the master instruction manual for building a Debian package (.deb).

Whenever a Linux distribution like Debian, Ubuntu, or Linux Mint needs to turn raw, upstream source code (like the EDK2 repository) into an easily installable package, the build servers look for this exact file to tell them what to do.

```
cd ~
apt-get source ovmf
cat edk2-*/debian/rules
```

The output below is the actual content of debian/rules from Ubuntu edk2 version 2025.11-3ubuntu7. This file is the ground truth for every flag in the Ubuntu OVMF binary.

Flag Definition Block (from debian/rules)

This is the section of debian/rules that defines the build flags for every OVMF variant:

```
PCD_RELEASE_DATE = $(shell date -d@$(SOURCE_DATE_EPOCH) "+%m/%d/%Y")
PCD_FLAGS = --pcd PcdFirmwareVendor=L"$(shell lsb_release -is) distribution of EDK II\0"
PCD_FLAGS += --pcd PcdFirmwareVersionString=L"$(DEB_VERSION)\0"
PCD_FLAGS += --pcd PcdFirmwareReleaseDateString=L"$(PCD_RELEASE_DATE)\0"
COMMON_FLAGS = -DCC_MEASUREMENT_ENABLE=TRUE
COMMON_FLAGS += -DNETWORK_HTTP_BOOT_ENABLE=TRUE
COMMON_FLAGS += -DNETWORK_IP6_ENABLE=TRUE
COMMON_FLAGS += -DNETWORK_TLS_ENABLE
COMMON_FLAGS += $(PCD_FLAGS)

NO_STRICTNX_COMMON_FLAGS = --pcd PcdUninstallMemAttrProtocol=TRUE

OVMF_COMMON_FLAGS = $(COMMON_FLAGS)
OVMF_COMMON_FLAGS += -DTPM2_ENABLE=TRUE
# Uncomment for in-band debug messages
# OVMF_COMMON_FLAGS += "-DDEBUG_ON_SERIAL_PORT=TRUE"
OVMF_4M_COMMON_FLAGS = $(OVMF_COMMON_FLAGS) -DFD_SIZE_4MB
OVMF_4M_NO_SECBOOT_FLAGS = $(OVMF_4M_COMMON_FLAGS) $(NO_STRICTNX_COMMON_FLAGS)
OVMF_4M_LEGACY_FLAGS = $(OVMF_4M_COMMON_FLAGS) $(NO_STRICTNX_COMMON_FLAGS) -DPVSCSI_ENABLE=TRUE -DMPT_SCSI_ENABLE=TRUE
OVMF_4M_SECBOOT_FLAGS = $(OVMF_4M_COMMON_FLAGS) -DBUILD_SHELL=FALSE -DSECURE_BOOT_ENABLE=TRUE -DSMM_REQUIRE=TRUE
OVMF_AMDSEV_FLAGS = $(OVMF_COMMON_FLAGS)
OVMF_4M_SECBOOT_CC_FLAGS = $(OVMF_4M_COMMON_FLAGS) -DBUILD_SHELL=FALSE -DSECURE_BOOT_ENABLE=TRUE -DSMM_REQUIRE=FALSE
OVMF_4M_SECBOOT_STRICTNX_FLAGS = $(OVMF_4M_COMMON_FLAGS) -DBUILD_SHELL=FALSE -DSECURE_BOOT_ENABLE=TRUE -DSMM_REQUIRE=TRUE
```

```
AAVMF_COMMON_FLAGS = $(COMMON_FLAGS)
AAVMF_COMMON_FLAGS += -DTPM2_ENABLE=TRUE
AAVMF_COMMON_FLAGS += -DTPM2_CONFIG_ENABLE=TRUE
AAVMF_COMMON_FLAGS += -DCAVIUM_ERRATUM_27456=TRUE
AAVMF_NO_SECBOOT_FLAGS = $(COMMON_FLAGS) $(NO_STRICTNX_COMMON_FLAGS)
AAVMF_SECBOOT_FLAGS = $(AAVMF_COMMON_FLAGS) -DBUILD_SHELL=FALSE -DSECURE_BOOT_ENABLE=TRUE
AAVMF_SECBOOT_STRICTNX_FLAGS = $(AAVMF_COMMON_FLAGS) -DBUILD_SHELL=FALSE -DSECURE_BOOT_ENABLE=TRUE

RISCV64_FLAGS = $(COMMON_FLAGS)
LOONGARCH64_FLAGS = $(COMMON_FLAGS)
```

Command for no-sec boot variant creation

```
build-ovmf-no-secboot: $(OVMF64_NO_SECBOOT_BUILD_DIR)/FV/OVMF.fd $(OVMF64_NO_SECBOOT_BUILD_DIR)/FV/OVMF_CODE.fd
build-efi-shell-x64: $(OVMF64_NO_SECBOOT_BUILD_DIR)/X64/Shell.efi
$(OVMF64_NO_SECBOOT_BUILD_DIR)/FV/OVMF.fd $(OVMF64_NO_SECBOOT_BUILD_DIR)/FV/OVMF_CODE.fd $(OVMF64_NO_SECBOOT_BUILD_DIR)/X64/Shell.efi &: debian/setup-build-stamp
$(call do_build,ovmf,no-secboot,X64,OvmfPkg/OvmfPkgX64.dsc,$(OVMF_4M_NO_SECBOOT_FLAGS))
```

4. How we created the OVMF from EDK2

Step1: Downloading EDK2 source code

```
git clone --recurse-submodules \
  https://github.com/tianocore/edk2.git
```

Step2: Compiling Base Tools

```
make -C BaseTools -j"${nproc}"
```

Step 3: Environment Setup

```
export PYTHON3_ENABLE=TRUE
export PYTHON_COMMAND=python3
# shellcheck source=/dev/null
source edksetup.sh --reconfig
```

Step 4: Define Shared Flag Sets

Ubuntu defines layered flag sets in debian/rules. These compose together — each variant inherits from the common base and adds its specific flags:

```
build \
  -a X64 \
  -t "${EDK2_GCC_TAG}" \
  -b RELEASE \
  -p OvmfPkg/OvmfPkgX64.dsc \
  -D FD_SIZE_4MB \
  -D CC_MEASUREMENT_ENABLE=TRUE \
  -D NETWORK_HTTP_BOOT_ENABLE=TRUE \
  -D NETWORK_IP6_ENABLE=TRUE \
  -D NETWORK_TLS_ENABLE \
  -D TPM2_ENABLE=TRUE \
  --pcd PcdUninstallMemAttrProtocol=TRUE \
  -n "${nproc}"
```

Step 5: Install to System Paths

The resulting .fd files are copied to /usr/share/OVMF/ with standardised names. Variable store templates (.snakeoil.fd, .ms.fd) have pre-enrolled Secure Boot keys injected using the virt-fw-vars tool.

Build Process -

```
[INFO] Building OVMF UEFI firmware from edk2 source...
Cloning into 'edk2'...
remote: Enumerating objects: 422910, done.
remote: Counting objects: 100% (513/513), done.
remote: Compressing objects: 100% (268/268), done.
remote: Total 422910 (delta 318), reused 246 (delta 245), pack-reused 422397 (from 4)
Receiving objects: 100% (422910/422910), 332.90 MiB | 3.66 MiB/s, done.
Resolving deltas: 100% (308400/308400), done.
Updating files: 100% (10015/10015), done.
Submodule 'BaseTools/Source/C/BrotliCompress/brotli' (https://github.com/google/brotli) registered for path 'BaseTools/Source/C/BrotliCompress/brotli'
Submodule 'CryptoPkg/Library/MbedTlsLib/mbedtls' (https://github.com/ARMmbed/mbedtls) registered for path 'CryptoPkg/Library/MbedTlsLib/mbedtls'
Submodule 'CryptoPkg/Library/OpensslLib/openssl' (https://github.com/openssl/openssl) registered for path 'CryptoPkg/Library/OpensslLib/openssl'
```

```
madhu@Madhur: /opt
[INFO] Compiling edk2 BaseTools...
make: Entering directory '/opt/cxl_workspace/tools/edk2/BaseTools'
make -C Source/C
make -C Source/Python
make[1]: Entering directory '/opt/cxl_workspace/tools/edk2/BaseTools/Source/Python'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/opt/cxl_workspace/tools/edk2/BaseTools/Source/Python'
make[1]: Entering directory '/opt/cxl_workspace/tools/edk2/BaseTools/Source/C'
/usr/bin/python3 ./Makefiles/GnuMakeUtils.py md .
/usr/bin/python3 ./Makefiles/GnuMakeUtils.py md ./libs
/usr/bin/python3 ./Makefiles/GnuMakeUtils.py md ./bin
Directory . already exists.
make -C Common
make[2]: Entering directory '/opt/cxl_workspace/tools/edk2/BaseTools/Source/C/Common'
gcc -c -I .. -I ../Include/Common -I ../Include/ -I ../Include/IndustryStandard -I ../Common/ -I .. -I . -I ../..
../MdePkg/Include/X64/ -I ../..../MdePkg/Include -MD -fshort-wchar -fno-strict-aliasing -fwrapv -fno-delete-n
ull-pointer-checks -Wall -Werror -Wno-deprecated-declarations -Wno-stringop-truncation -Wno-restrict -Wno-unused-res
ult -nostdlib -g -O2 BasePeCoff.c -o BasePeCoff.o
gcc -c -I .. -I ../Include/Common -I ../Include/ -I ../Include/IndustryStandard -I ../Common/ -I .. -I . -I ../..
../MdePkg/Include/X64/ -I ../..../MdePkg/Include -MD -fshort-wchar -fno-strict-aliasing -fwrapv -fno-delete-n
ull-pointer-checks -Wall -Werror -Wno-deprecated-declarations -Wno-stringop-truncation -Wno-restrict -Wno-unused-res
ult -nostdlib -g -O2 BinderFuncs.c -o BinderFuncs.o
gcc -c -I .. -I ../Include/Common -I ../Include/ -I ../Include/IndustryStandard -I ../Common/ -I .. -I . -I ../..
../MdePkg/Include/X64/ -I ../..../MdePkg/Include -MD -fshort-wchar -fno-strict-aliasing -fwrapv -fno-delete-n
ull-pointer-checks -Wall -Werror -Wno-deprecated-declarations -Wno-stringop-truncation -Wno-restrict -Wno-unused-res
ult -nostdlib -g -O2 CommonLib.c -o CommonLib.o
gcc -c -I .. -I ../Include/Common -I ../Include/ -I ../Include/IndustryStandard -I ../Common/ -I .. -I . -I ../..
../MdePkg/Include/X64/ -I ../..../MdePkg/Include -MD -fshort-wchar -fno-strict-aliasing -fwrapv -fno-delete-n
ull-pointer-checks -Wall -Werror -Wno-deprecated-declarations -Wno-stringop-truncation -Wno-restrict -Wno-unused-res
ult -nostdlib -g -O2 Crc32.c -o Crc32.o
```

```
madhu@Madhur: /opt
test_Common_BuildVersion (CheckPythonSyntax.Tests.test_Common_BuildVersion) ... ok
test_Common_DataType (CheckPythonSyntax.Tests.test_Common_DataType) ... ok
test_Common_Edk2_Capsule_FmpPayloadHeader (CheckPythonSyntax.Tests.test_Common_Edk2_Capsule_FmpPayloadHeader) ... ok
test_Common_Edk2_Capsule___init__ (CheckPythonSyntax.Tests.test_Common_Edk2_Capsule___init__) ... ok
test_Common_Edk2___init__ (CheckPythonSyntax.Tests.test_Common_Edk2___init__) ... ok
test_Common_EdkLogger (CheckPythonSyntax.Tests.test_Common_EdkLogger) ... ok
test_Common_Expression (CheckPythonSyntax.Tests.test_Common_Expression) ... ok
test_Common_GlobalData (CheckPythonSyntax.Tests.test_Common_GlobalData) ... ok
test_Common_LongFilePathOs (CheckPythonSyntax.Tests.test_Common_LongFilePathOs) ... ok
test_Common_LongFilePathOsPath (CheckPythonSyntax.Tests.test_Common_LongFilePathOsPath) ... ok
test_Common_LongFilePathSupport (CheckPythonSyntax.Tests.test_Common_LongFilePathSupport) ... ok
test_Common_Misc (CheckPythonSyntax.Tests.test_Common_Misc) ... ok
test_Common_MultipleWorkspace (CheckPythonSyntax.Tests.test_Common_MultipleWorkspace) ... ok
test_Common_Parsing (CheckPythonSyntax.Tests.test_Common_Parsing) ... ok
test_Common_RangeExpression (CheckPythonSyntax.Tests.test_Common_RangeExpression) ... ok
test_Common_StringUtils (CheckPythonSyntax.Tests.test_Common_StringUtils) ... ok
test_Common_TargetTxtClassObject (CheckPythonSyntax.Tests.test_Common_TargetTxtClassObject) ... ok
test_Common_ToolDefClassObject (CheckPythonSyntax.Tests.test_Common_ToolDefClassObject) ... ok
test_Common_Uefi_Capsule_CapsuleDependency (CheckPythonSyntax.Tests.test_Common_Uefi_Capsule_CapsuleDependency) ...
ok
test_Common_Uefi_Capsule_FmpAuthHeader (CheckPythonSyntax.Tests.test_Common_Uefi_Capsule_FmpAuthHeader) ... ok
test_Common_Uefi_Capsule_FmpCapsuleHeader (CheckPythonSyntax.Tests.test_Common_Uefi_Capsule_FmpCapsuleHeader) ... ok
test_Common_Uefi_Capsule_UefiCapsuleHeader (CheckPythonSyntax.Tests.test_Common_Uefi_Capsule_UefiCapsuleHeader) ...
ok
test_Common_Uefi_Capsule___init__ (CheckPythonSyntax.Tests.test_Common_Uefi_Capsule___init__) ... ok
test_Common_Uefi___init__ (CheckPythonSyntax.Tests.test_Common_Uefi___init__) ... ok
test_Common_VariableAttributes (CheckPythonSyntax.Tests.test_Common_VariableAttributes) ... ok
test_Common_VpdInfoFile (CheckPythonSyntax.Tests.test_Common_VpdInfoFile) ... ok
test_Common___init__ (CheckPythonSyntax.Tests.test_Common___init__) ... ok
test_Common_caching (CheckPythonSyntax.Tests.test_Common_caching) ... ok
```

CEDT Table Verification -

```
root@ubuntu:/tmp# cat /tmp/CEDT.dsl
/*
 * Intel ACPI Component Architecture
 * AML/ASL+ Disassembler version 20230628 (64-bit version)
 * Copyright (c) 2000 - 2023 Intel Corporation
 *
 * Disassembly of CEDT, Sun Jun 14 13:08:54 2026
 *
 * ACPI Data Table [CEDT]
 *
 * Format: [HexOffset DecimalOffset ByteLength]  FieldName : FieldValue (in hex)
 */

[000h 0000 004h]                Signature : "CEDT"          [CXL Early Discovery Table]
[004h 0004 004h]                Table Length : 0000006C
[008h 0008 001h]                Revision : 01
[009h 0009 001h]                Checksum : 77
[00Ah 0010 006h]                Oem ID : "BOCHS "
[010h 0016 008h]                Oem Table ID : "BXP" "
[018h 0024 004h]                Oem Revision : 00000001
[01Ch 0028 004h]                Asl Compiler ID : "BXP"
[020h 0032 004h]                Asl Compiler Revision : 00000001

[024h 0036 001h]                Subtable Type : 00 [CXL Host Bridge Structure]
[025h 0037 001h]                Reserved : 00
[026h 0038 002h]                Length : 0020
[028h 0040 004h]                Associated host bridge : 0000000C
[02Ch 0044 004h]                Specification version : 00000001
[030h 0048 004h]                Reserved : 00000000
```

```

[025h 0037 001h]          Reserved : 00
[026h 0038 002h]          Length : 0020
[028h 0040 004h]    Associated host bridge : 0000000C
[02Ch 0044 004h]    Specification version : 00000001
[030h 0048 004h]          Reserved : 00000000
[034h 0052 008h]    Register base : 0000000580000000
[03Ch 0060 008h]    Register length : 0000000000010000

[044h 0068 001h]    Subtable Type : 01 [CXL Fixed Memory Window Structure]
[045h 0069 001h]          Reserved : 00
[046h 0070 002h]          Length : 0028
[048h 0072 004h]          Reserved : 00000000
[04Ch 0076 008h]    Window base address : 0000000590000000
[054h 0084 008h]    Window size : 0000000100000000
[05Ch 0092 001h]    Interleave Members (2^n) : 00
[05Dh 0093 001h]    Interleave Arithmetic : 00
[05Eh 0094 002h]          Reserved : 0000
[060h 0096 004h]    Granularity : 00000004
[064h 0100 002h]    Restrictions : 000F
[066h 0102 002h]          QtgId : 0000
[068h 0104 004h]    First Target : 0000000C

```

Raw Table Data: Length 108 (0x6C)

```

0000: 43 45 44 54 6C 00 00 00 01 77 42 4F 43 48 53 20 // CEDTl....wBOCHS
0010: 42 58 50 43 20 20 20 20 01 00 00 00 42 58 50 43 // BXPC      ....BXPC
0020: 01 00 00 00 00 00 20 00 0C 00 00 00 01 00 00 00 // .....
0030: 00 00 00 00 00 00 00 80 05 00 00 00 00 00 01 00 // .....
0040: 00 00 00 00 01 00 28 00 00 00 00 00 00 00 00 90 // .....(.....
0050: 05 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00 // .....

```